User Manual: PWApp

Lukas Weissinger*, Simon Hubmer†, Ronny Ramlau*†, Henning U. Voss‡

December 4, 2025

1 Introduction

This manual provides comprehensive instructions for installing and using the MATLAB-based **PWApp** available at `https://github.com/Weisslu/PWApp`, a tool designed for scientific research on pulse wave velocity (PWV) estimation from MRI-based blood flow velocity data. It also explains the mathematical methods implemented in the app.

1.1 Purpose of the App

The goal of the PWApp is to offer a complete solution for PWV estimation (and pulse wave splitting, see Section 4.1.5) using blood flow velocity data from 2D or 4D flow MRI. Key features of the app include:

- **Data loading:** MRI data is loaded via an external script. Example scripts are available in the GitHub repository. Custom scripts can be used to account for the variability of different datasets.
- **Optional 3D angiogram input:** A 3D angiogram can optionally be loaded to assist in the calculation of arterial length.
- **Manual datapoint selection:** An option to manually select the waveforms which are to be considered in the PWV estimation.
- **Multiple PWV estimation methods:** Several methods are available for PWV estimation (see Section 4.1), and their results can be compared within the app.

*Johann Radon Institute Linz, Altenbergerstraße 69, A-4040 Linz, Austria, (lukas.weissinger@ricam.oeaw.ac.at, ronny.ramlau@ricam.oeaw.ac.at).

†Johannes Kepler University Linz, Institute of Industrial Mathematics, Altenbergerstraße 69, A-4040 Linz, Austria, (simon.hubmer@jku.at, ronny.ramlau@jku.at).

‡Cornell University - College of Human Ecology, Cornell MRI Facility, Martha Van Rensselaer Hall, Ithaca NY 14853, United States of America, (hv28@cornell.edu)

2 Installation

PWApp is available both as MATLAB source-code and as a standalone executable (.exe). MATLAB Runtime is required to run the app and can be downloaded from: `https://de.mathworks.com/products/compiler/MATLAB-runtime.html`. Note that the app installation file includes MATLAB Runtime if it is not already installed on the machine. PWApp was developed using MATLAB R2025a, and thus compatibility with older MATLAB versions is not guaranteed. Once installed, the .exe file can be launched directly from the installation folder.

3 User Interface Overview

The app layout is shown in Figure 3.1. The main window is separated into two panels:

- **Options panel (left)**: for input settings, parameters, and actions
- **Visualization panel (right)**: for displaying outputs, previews, and results

To perform a complete PWV estimation from raw MRI data, the app follows a structured five-step workflow which has to be completed in order. The steps are:

1. **Data loading**
2. **Centerline extraction and branch identification**
3. **Data-point selection**
4. **Waveform computation**
5. **PWV estimation**

Again, note that these steps must be completed consecutively. Interface elements for later steps only become available and visible once the preceding steps have been completed.

All relevant controls and settings for each step are found in the **options panel**, while the **visualization panel** displays results and allows control over aspects such as viewing perspectives.

4 Functionality of PWApp

Step 1: Load flow MRI data

PWApp supports both 2D and 4D flow MRI data. Due to variability in metadata across scanners and imaging methods, data must be loaded via a custom script. This script is selected using the "Load velocity data" button in the app. The script must return the variables `[cardiacFrames, dim, res, delays, phase, magnitude, ori, pos,`

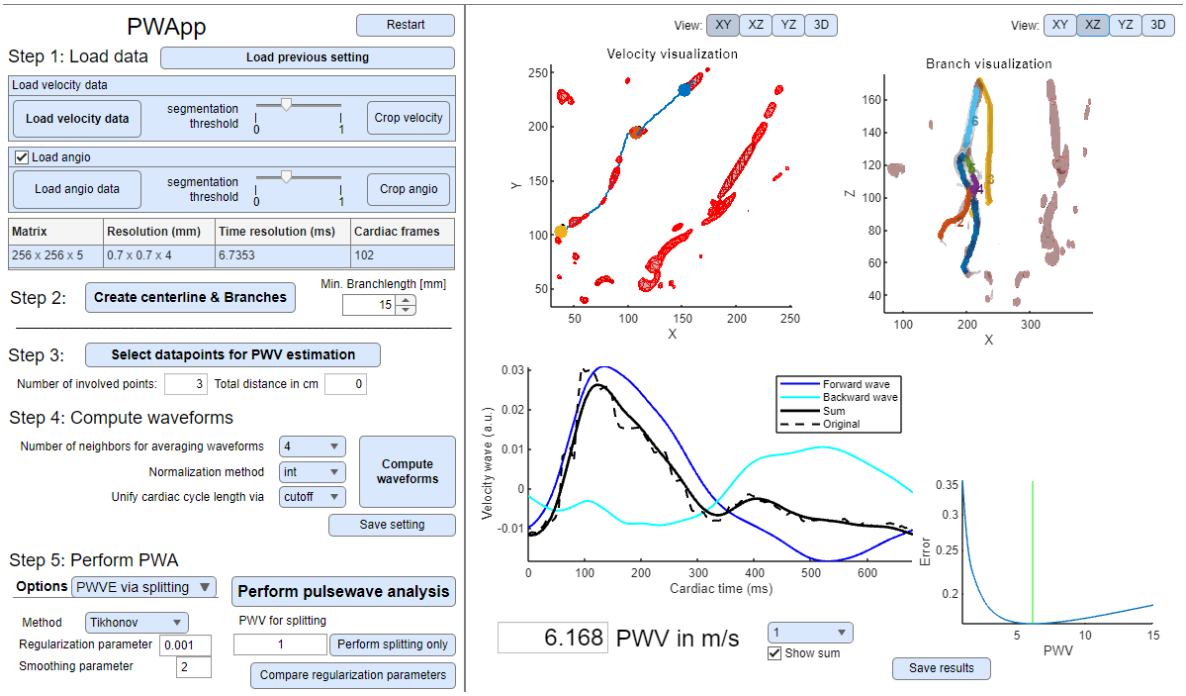

Figure 3.1: Main layout after appearance of all features.

`directionFlag]` in order, as listed in Table 4.1. An example script and dataset is available in the GitHub repository `https://github.com/Weisslu/PWApp`.

After loading, the app automatically segments flowing and non-flowing voxels using an approach based on [1]. The segmentation threshold can be manually adjusted. Higher thresholds classify more voxels as flowing. Flowing voxels are visualized in red in the top-left plot of the visualization panel, see Figure 3.1. Subsequently, the dataset can be cropped using the "Crop velocity" button. For best visualization and analysis results, crop to the smallest possible volume which still includes all relevant artery segments.

Load 3D Angiogram (Optional): Optionally, a 3D angiogram can be loaded to assist with arterial length computation, especially when:

- Flow data only partially covers artery segments,
- High-resolution length estimation is required.

The angiogram is also loaded via a script, selected using the "Load angio data" button. The script must return the variables `[dim, res, angio, ori, pos, directionFlag]` in order, as listed in Table 4.2. An example script is also available in the GitHub repository `https://github.com/Weisslu/PWApp`. After loading, the angiogram can be cropped and the segmentation threshold can be changed, similarly to the flow data. Segmentation results appear in the top right plot of the visualization panel. The view angle can be changed using the buttons above the plot.

Variable	Size	Description
$tdim$	int	number of images per cardiac cycle (temporal dimension)
$dim = [xdim, ydim, zdim]$	$1 \cdot 3$ int array	Spatial dimensions of the image volume in x, y, and z
$res = [xres, yres, zres]$	$1 \cdot 3$ float array	Voxel size (mm) in x,y,z directions
$delays$	$zdim \cdot tdim$ float array	Time delay values (ms) for every slice
$phase$	$xdim \cdot ydim \cdot zdim \cdot tdim$ float array	Phase image volume
$magnitude$	$xdim \cdot ydim \cdot zdim \cdot tdim$ float array	Magnitude image volume
$ori = [ori_x, ori_y]$	$1 \cdot 6$ float array	two 3D vectors indicating x- and y-axis orientation
$pos = [pos_1, pos_2, pos_3]$	$1 \cdot 3$ float array	Position of first pixel in the first slice
$directionFlag$	$\{-1, +1\}$	$+1$ if $ori_x \times ori_y = ori_z$, -1 if $ori_x \times ori_y = -ori_z$

Table 4.1: Required variables for flow MRI data.

Variable	Size	Description
$[xdim, ydim, zdim]$	$1 \cdot 3$ int array	Spatial dimensions of the angiogram volume
$[xres, yres, zres]$	$1 \cdot 3$ float array	Voxel size (mm) in x,y,z directions
$angio$	$xdim \cdot ydim \cdot zdim$ float array	Angiographic intensity values
$[ori_x, ori_y]$	$1 \cdot 6$ float array	Two 3D vectors indicating x- and y-axis orientation
$[pos_1, pos_2, pos_3]$	$1 \cdot 3$ float array	Position of the first pixel in the first slice
$directionFlag$	$\{-1, +1\}$	$+1$ if $ori_x \times ori_y = ori_z$, -1 if $ori_x \times ori_y = -ori_z$

Table 4.2: Required variables for angiogram data.

Cropping interface: The "Crop" button opens a pop-up window (see Figure 4.1), displaying maximum intensity projections (MIPs) of the 3D datasets from multiple angles. Each view includes its own cropping option. Clicking the crop button on a view opens a new window where a rectangular region of interest (ROI) can be drawn. Voxels

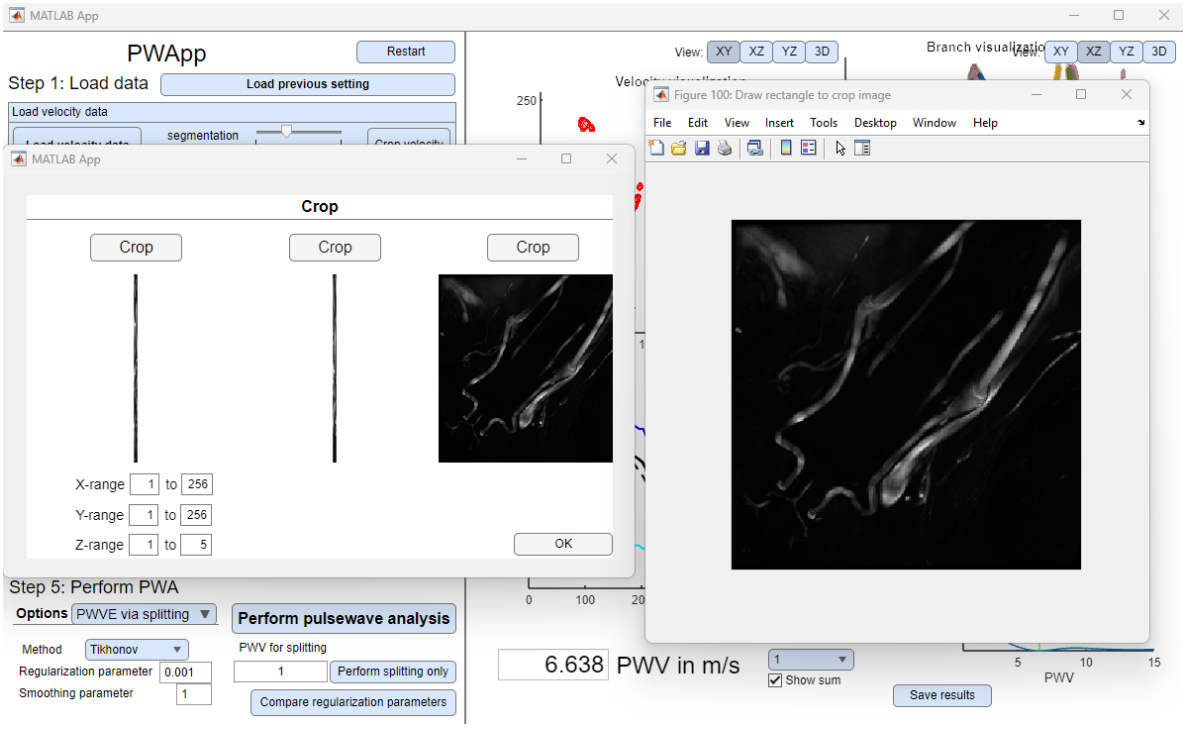

Figure 4.1: Pop-up window to crop the data.

outside the ROI will be removed from the dataset.

Step 2: Centerline extraction and branch identification

After data loading, cropping, and threshold adjustment, a centerline is computed for the segmented artery data. This step also separates the artery into individual branches. Note that:

- If a 3D angiogram has been loaded, the centerline is computed based on the angiographic data.
- The used algorithm is based on [20] and extended from [19].
- Additionally, a visual overlay of the angiographic and velocity data is generated automatically using image orientation and position information from metadata.

The computed branches and overlay are displayed in the top-right plot of the visualization panel.

Step 3: Select data-points for PWV estimation

Click the "Select data-points for PWV estimation"-button to open a pop-up window. Here, spatial points along the centerlines of specific branches (see Figure 4.2) can be manually selected via directly clicking onto specific points in the left figure.

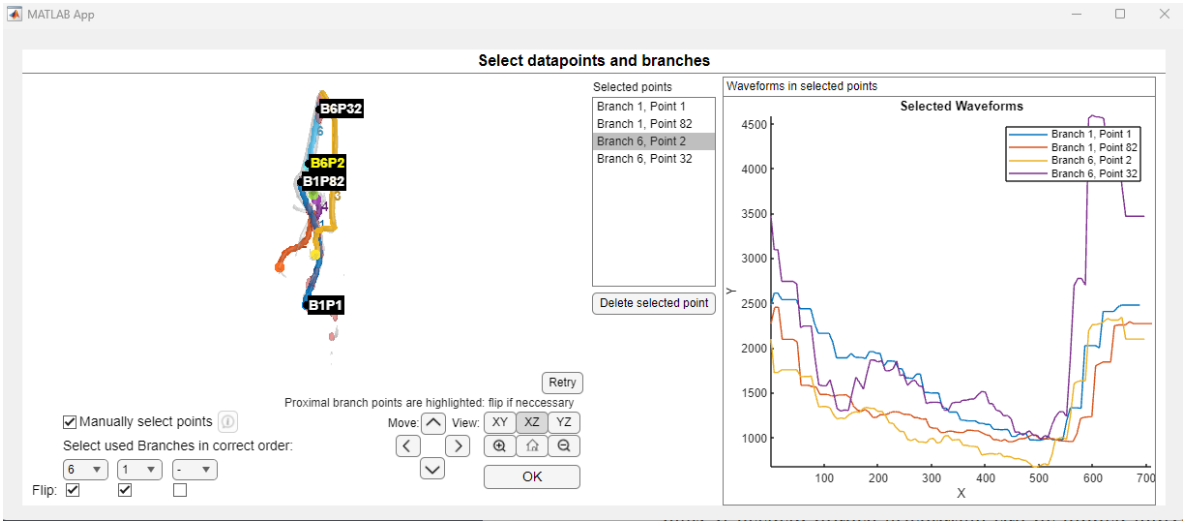

Figure 4.2: Data point selection window, which allows manual selection of spatial points along the centerlines of artery branches to be used in PWV estimation.

- Each branch is color-coded and numbered for clarity.
- Selecting a point displays its corresponding waveform in the right figure.
- Points can be deleted from the selection again.
- Select all branches which connect the chosen points to define the full artery segment used for PWV estimation.
- Correct branch order and orientation must be selected. An example illustrating the proper selecting principles is provided in Figure 4.3.
- Selected branches must form a connected artery segment.
- Each branch has a default orientation: the proximal point is highlighted in the plot. If needed, branch orientation can be flipped individually.

Automatic Data Point Selection (Optional): Instead of selecting individual points, all available data points along a branch can be selected automatically:

- Select the desired branches.
- Uncheck the "Manual selection" checkbox.
- Clicking on a point in the "Selected points" list highlights the corresponding point in the left figure.

The view angle and field of view of the left figure can be changed via the "Move" and "View" buttons. Distances between selected points are computed using a Bezier-curve

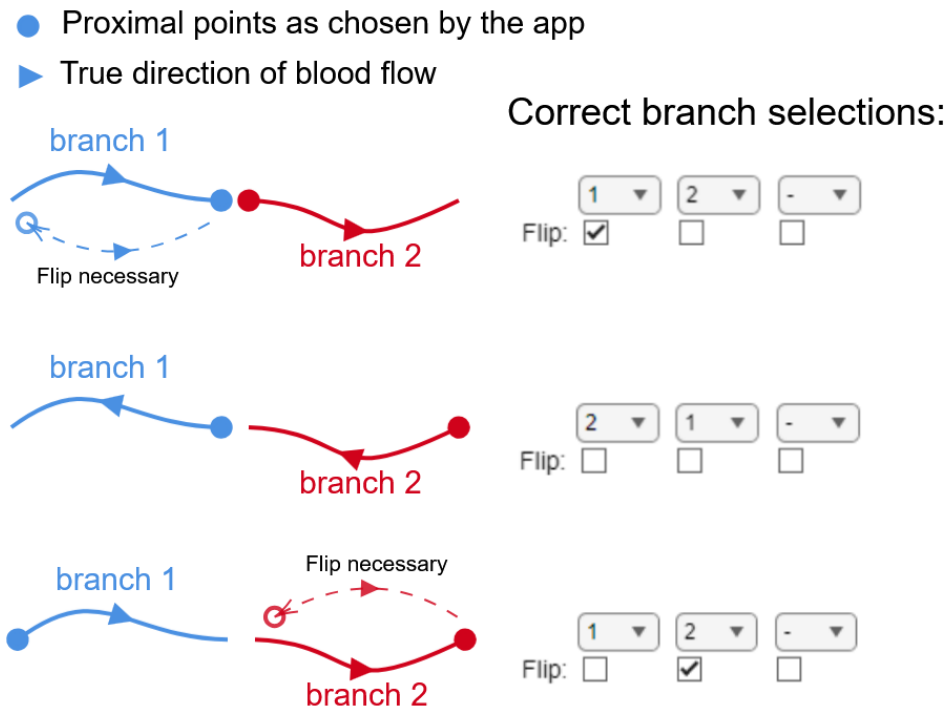

Figure 4.3: Examples on how to select branch order and orientation. If the proximal point is shown at the end of the artery segment, the corresponding "Flip" checkbox has to be checked. The direction of blood flow has to be known by the user.

approximation (De Casteljau's algorithm, see [6]). This achieves sub-pixel accuracy and mimics the natural curvature of arteries. The resulting smoothed artery segment is shown in the bottom-right plot of the visualization panel. One may use this visualization to verify correct branch connection and orientation.

Step 4: Compute waveforms

Before performing PWV estimation, the waveforms selected in Step 3 may require pre-processing. This is necessary to improve signal quality and ensure consistent comparison across locations. In particular, the **partial voluming effect** (see [17]) might reduce the amplitude of flow waveforms.

As a remedy, the blood flow velocity data is commonly used to compute flow waveforms along several artery cross-sections. However, if the number of voxels spanning over the cross-section is rather small, the flow signal may become noisy or inaccurate. To mitigate this, we propose wavefront normalization [7]. Use the drop-down menu in the app to select a normalization method from the following list of available options:

For technical details, see MATLAB's `normalize` function documentation. To further reduce noise, waveforms can be averaged over a user-defined number of neighboring centerline points.

Method	Description
int (recommended)	Simulates blood flow in arbitrary units. Each waveform is L1-normalized (sum of values is equal).
range	Scales each waveform to fit within the interval [0, 1].
L2	Normalizes waveforms using the Euclidean (L2) norm.
scale	Scales each waveform by its standard deviation.

In some cases, cardiac cycle lengths may vary across different image slices, depending on the MRI acquisition settings. To correct for this, two options are available:

Method	Description
cutoff (recommended)	Aligns the foot of the waveform to the beginning of the cardiac cycle. Waveforms are trimmed to match the shortest available cycle length. Missing values are interpolated to ensure all waveforms have equal (vector) length.
average	Retains original waveform lengths and uses the average cycle length for alignment, without trimming or interpolation.

Once computed, the processed waveforms are displayed in the bottom-left plot of the visualization panel.

Step 5: Perform PWV estimation

Once waveforms have been preprocessed, you can proceed to estimate the PWV. Select the desired estimation method from the drop-down list and click "Perform pulsewave analysis" to run the analysis. The app provides several estimation methods, explained in detail in the next section. Note that visualization in the bottom plots of the visualization panel differs for each method, as explained in the corresponding section below.

4.1 PWV estimation methods

PWV estimation has been extensively researched [12, 9, 15, 21], and multiple methods have been proposed for MRI-based flow data [11, 10, 13, 8, 2, 4, 16]. The PWApp includes five different methods for PWF estimation, which are described in the following subsections. References are provided if you want to explore the theoretical background or implementation details in depth. The supported PWV estimation methods are:

- Time-to-Foot (TTF) [13]
- Cross-Correlation [8]
- Wavelet [2]
- Maximum Likelihood Estimation [4]
- Pulse Wave Splitting (Inverse Problem Approach) [24]

4.1.1 Time-to-foot (TTF) method

Based on [13], a line is fitted to the upslope portion of the waveform (between 20% and 80% of the peak flow). The TTF time-shift for each waveform is then defined as the intersection of the fitted line with zero. A linear fit is applied to the plot of TTF vs. spatial position, and the PWV is calculated as the inverse of the slope of the linear fit. The bottom-left plot shows the TTF across locations and the fitted line.

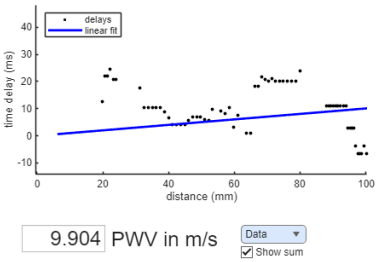

Figure 4.4: Example result plot for the TTF method.

4.1.2 Cross-correlation method

Based on [8], this method uses cross-correlation to determine time shifts between waveforms. The waveform at each spatial point at the centerline is compared to the waveform at the first point using a cross-correlation function. The cross-correlation function applies a time-shift to the more distal waveform until the highest correlation value between the two waveforms is obtained. This effectively returns a time shift for each location relative to the first location. A line is then fitted to the plot of spatial location over time shifts, and the inverse of the slope is taken as the PWV estimate. The bottom-left plot shows the time shifts across locations and the fitted line.

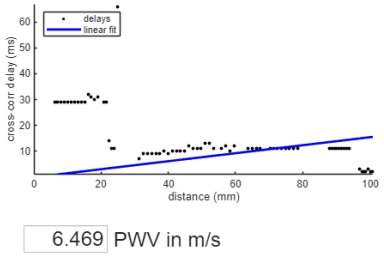

Figure 4.5: Example result plot for the cross-correlation method.

4.1.3 Wavelet method

Based on [2], this method performs a cross-correlation similarly to the cross-correlation method, but for wavelet-transformed signals. Benefits of transforming into frequency domains have been shown in [14]. One of them is the time-localization of the wavelet-transform, which is leveraged to restrict the signal to the systolic upstroke only, in order to reduce the impact of reflected (backwards traveling) parts. The source-code for this method in the PWApp is from [19, 20]. The bottom-left plot shows the time shifts across locations and the fitted line.

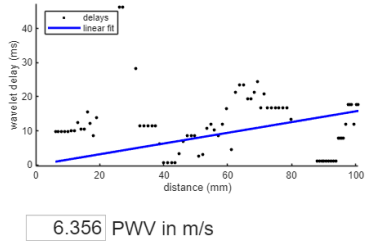

Figure 4.6: Example result plot for the wavelet method.

4.1.4 Maximum-likelihood estimator

Based on [4], this method assumes that all observed waveforms are identical but phase-shifted due to different propagation delays. Let $p_i(t)$ denote the (normalized) velocity

wave-form obtained at position i, u the PWV, and L_i the arterial distance between the first measurement point and the point i, where $i = 1, \ldots, N$. The cardiac cycle is characterized via the time-points t_j, $j = 1, \ldots, M$, such that the solution for p_1, u depending on the data $p_{ij} = p_i(t_j)$, is given by

$$(p_1, u) = \arg\min_{p,v} \sum_{i=1}^{N} \sum_{j=1}^{M} \left(p(t_j - L_i/u) - p_{ij}\right)^2 . \tag{4.1}$$

The bottom left plot in the visualization panel shows the initial guess for the waveform (all waveforms averaged), and the final reconstruction of the waveform.

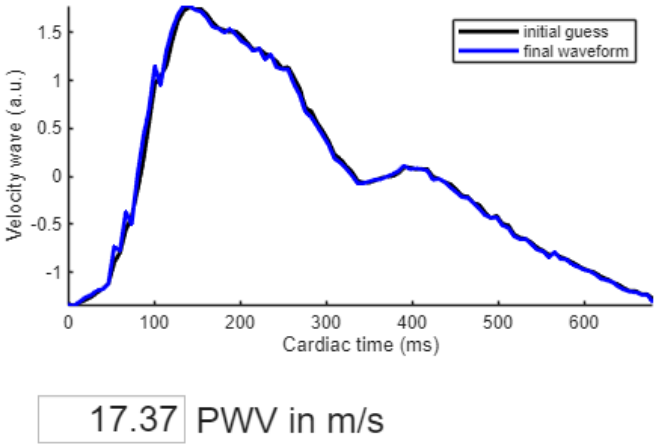

Figure 4.7: Example result plot for the maximum-likelihood method.

4.1.5 Pulse wave splitting

This method is based on [24] and, in contrast to the methods introduced before, models the pulse waves p_i in each spatial point i as a superposition of a forward and backward traveling part:

$$p_i(t) = p_{if} + p_{ib} .$$

These component waves are assumed to be identical but phase shifted, i.e., $p_{if}(t) = p_{1f}(t - L_i/u)$ and $p_{ib}(t) = p_{1b}(t + L_i/u)$. This set of equations leads to an ill-posed inverse problem and can be solved using methods from regularization theory for the unknowns $\hat{p}_{1f}, \hat{p}_{Nb}, u$. Regularization methods available in the PWApp are Tikhonov regularization [5, 18] and FISTA [3]. Additionally, a constraint ("+ bw cond") can be added to the reconstruction methods, stating that the backward wave has to be smaller or equal than the forward wave in each frequency component, cf. [22].

For this method, you must set the **regularization** and **smoothing** parameters: The higher those parameters, the smoother the waves. If you are unsure on how to choose these parameters, we advise to keep the default values. For FISTA, the regularization parameter equals the percentage of cut-off frequency components.

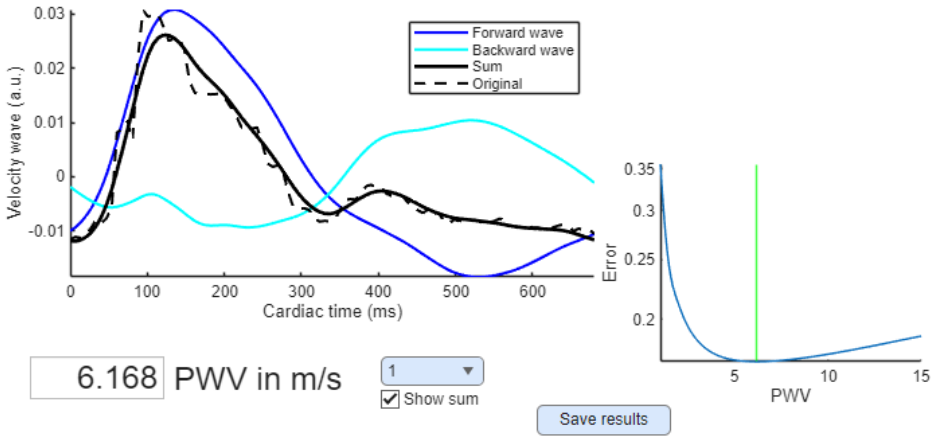

Figure 4.8: Example result plot for the splitting method.

The bottom-left plot shows the separate forward and backward part of the wave, and optionally the resulting total wave compared to the original waveform. The bottom-right plot shows error values for different PWVs and the computed minimum as a green line, see also [22, 24]. Note that unlike the other methods, pulse wave splitting performs well even with few data points (as few as 3). Adding more waveforms does not necessarily improve accuracy, although averaging over neighbors (Step 4) is still advised to reduce noise.

Compare regularization parameters: This additional feature enables the comparison of the effects of different regularization parameters. Residual errors and the reconstructed PWVs for a specified range of regularization parameters are computed to help choose parameters and check for robustness of the method.

4.2 Reproducibility and result saving

To allow for reproducibility, PWApp allows you to save and reload specific configurations in a .mat file via the "save setting" button. Saving is available after performing data-point selection (Step 3). These saved settings can later be restored via the "load previous setting" button. Once PWV estimation has been performed, you can export the final results using the "Save results" button. This generates a .mat file containing:

- PWV
- All parameters used for the PWV estimation
- Velocity waveforms
- Corresponding distances along the vessel
- Forward and backward split waves (if pulse wave splitting was used)

5 Example

An example dataset is available in the GitHub repository. This dataset is the same as the one used in [24], and also appears in the figures throughout this manual. To load the example data you have two options:

- **Option 1:** Load from raw DICOMs

  1. Choose the script "read_dicoms.m" in data loading
  2. In the file selector dialog, navigate to the folder "example/flow_dicoms"
  3. (Optional) To load angiography data, use the script "read_angio_dicoms.m" and select a file from "example/angio_dicoms"

- **Option 2:** Load a preconfigured setting

  Use the "Load previous setting" button and select the file "example/savedex.mat". This will load a pre-processed version of the example data, including previously selected parameters and configurations.

6 Disclaimer

The MATLAB tool described here is an extension of the "4D Flow PWV Tool" developed by Eric Schrauben [19, 20], specifically tailored to the needs of the methods developed in [24] and problems arising in PWV estimation in the human brain. In contrast to Schrauben's tool, no flow computation is performed, which includes the computation of the area of the arterial cross-sections. Instead, we propose normalization of the waveforms, which leads to a pulse-wave (of an arbitrary unit) equivalent to the flow-wave. Further extensions include the visual selection of data-points and the possibility to include angiographic data for distance computation, which allows for the computation of the PWV on artery-segments with no available continuous velocity data. Lastly, the method [23] based on the inverse problems approach for pulse wave splitting is included in PWApp, which was the main motivation for developing this app.

HUV is an inventor on Cornell University patents related to this research.

7 Acknowledgments

HUV acknowledges support from the Nancy M. and Samuel C. Fleming Research Scholar Award in Intercampus Collaborations, Cornell University. RR and SH were funded in part by the Austrian Science Fund (FWF) SFB 10.55776/F68 "Tomography Across the Scales", project F6805-N36 (Tomography in Astronomy). For open access purposes, the authors have applied a CC BY public copyright license to any author-accepted manuscript version arising from this submission. LW is partially supported by the State of Upper Austria.

References

[1] A. G. Anderson, K. M. Johnson, J. Bock, M. Markl, and O. Wieben. "Comparison of image reconstruction algorithms for the depiction of vessel anatomy in PC VIPR datasets." In: *In Proc Intl Soc Mag Reson Med* 16 (2008), p. 934.

[2] I. Bargiotas et al. "Estimation of aortic pulse wave transit time in cardiovascular magnetic resonance using complex wavelet cross-spectrum analysis". In: *Journal of Cardiovascular Magnetic Resonance* 17.1 (Jan. 2015), p. 65. ISSN: 1097-6647. DOI: https://doi.org/10.1186/s12968-015-0164-7.

[3] A. Beck and M. Teboulle. "A Fast Iterative Shrinkage-Thresholding Algorithm for Linear Inverse Problems." In: *SIAM J. Imaging Sci.* 2.1 (2009), pp. 183–202. DOI: 10.1137/080716542.

[4] C. Björnfot et al. "Assessing cerebral arterial pulse wave velocity using 4D flow MRI". In: *Journal of Cerebral Blood Flow & Metabolism* 41.10 (2021). PMID: 33853409, pp. 2769–2777. DOI: 10.1177/0271678X211008744.

[5] H. W. Engl, M. Hanke, and A. Neubauer. *Regularization of inverse problems.* English. Dordrecht: Kluwer Academic Publishers, 1996, pp. viii + 321. ISBN: 0-7923-4157-0.

[6] G. Farin and D. Hansford. *The Essentials of CAGD.* CRC Press, 2000. ISBN: 9781439864111. URL: https://books.google.at/books?id=ODFRDwAAQBAJ.

[7] N. Feilmaier. *Denoising of the pulse wave in the human brain: A Sobolev embedding approach.* eng. 2024. URL: https://resolver.obvsg.at/urn:nbn:at:at-ubl:1-77149.

[8] S. W. Fielden et al. "A new method for the determination of aortic pulse wave velocity using cross-correlation on 2D PCMR velocity data". In: *Journal of Magnetic Resonance Imaging* 27.6 (May 2008), pp. 1382–1387. ISSN: 1522-2586. DOI: 10.1002/jmri.21387.

[9] C. A. Giller and D. R. Aasli. "Estimates of pulse-wave velocity and measurement of pulse transit-time in the human cerebral-circulation". In: *Ultrasound in Medicine and Biology* 20.2 (1994), pp. 101–105. ISSN: 0301-5629. DOI: Doi10.1016/0301-5629(94)90074-4.

[10] S. Hubmer, A. Neubauer, R. Ramlau, and H. U. Voss. "A conjugate-gradient approach to the parameter estimation problem of magnetic resonance advection imaging". In: *Inverse Problems in Science and Engineering* 28.8 (2020), pp. 1154–1165. DOI: 10.1080/17415977.2019.1708911.

[11] S. Hubmer, A. Neubauer, R. Ramlau, and H. U. Voss. "On the parameter estimation problem of magnetic resonance advection imaging". In: *Inverse Problems and Imaging* 12.1 (2018), pp. 175–204. DOI: 10.3934/ipi.2018007.

[12] J. K. J. Li. *Dynamics of the Vascular System.* Series on Bioengineering and Biomedical Engineering. River Edge, N.J.: World Scientific, 2004, xii, 257 p. ISBN: 9810249071.

[13] M. Markl et al. "Estimation of global aortic pulse wave velocity by flow-sensitive 4D MRI". In: *Magnetic Resonance in Medicine* 63.6 (Apr. 2010), pp. 1575–1582. ISSN: 1522-2594. DOI: https://doi.org/10.1002/mrm.22353.

[14] A. Meloni, H. Zymeski, A. Pepe, M. Lombardi, and J. C. Wood. "Robust estimation of pulse wave transit time using group delay". In: *Journal of Magnetic Resonance Imaging* 39.3 (2014), pp. 550–558. DOI: https://doi.org/10.1002/jmri.24207.

[15] S. I. Rabben et al. "An ultrasound-based method for determining pulse wave velocity in superficial arteries". In: *Journal of Biomechanics* 37.10 (2004), pp. 1615–1622. ISSN: 0021-9290. DOI: https://doi.org/10.1016/j.jbiomech.2003.12.031.

[16] L. A. Rivera-Rivera et al. "Assessment of vascular stiffness in the internal carotid artery proximal to the carotid canal in Alzheimer's disease using pulse wave velocity from low rank reconstructed 4D flow MRI". In: *Journal of Cerebral Blood Flow & Metabolism* 41.2 (Mar. 2020), pp. 298–311. DOI: 10.1177/0271678x20910302.

[17] S. A. Röll, A. C. F. Colchester, P. E. Summers, and L. D. Griffin. "Intensity-Based Object Extraction from 3D Medical Images Including a Correction of Partial Volume Errors." In: *BMVC*. 1994, pp. 1–10.

[18] O. Scherzer, M. Grasmair, H. Grossauer, M. Haltmeier, and F. Lenzen. *Variational Methods in Imaging*. Applied Mathematical Sciences. Springer New York, 2008. ISBN: 9780387692777.

[19] E. Schrauben. *4D Flow PWV Tool*. URL: https://github.com/schrau24/4DFlowPWVTool.

[20] E. Schrauben et al. "Fast 4D flow MRI intracranial segmentation and quantification in tortuous arteries". In: *Journal of Magnetic Resonance Imaging* 42.5 (Apr. 2015), pp. 1458–1464. ISSN: 1522-2586. DOI: 10.1002/jmri.24900.

[21] S. Vulliémoz, N. Stergiopulos, and R. Meuli. "Estimation of local aortic elastic properties with MRI". In: *Magnetic Resonance in Medicine* 47.4 (2002), pp. 649–654. DOI: https://doi.org/10.1002/mrm.10100.

[22] L. Weissinger. *Frame- and fourier-based reconstruction methods for pulse wave analysis and atmospheric tomography*. eng. 2025. URL: https://resolver.obvsg.at/urn:nbn:at:at-ubl:1-89632.

[23] L. Weissinger, S. Hubmer, R. Ramlau, and H. U. Voss. *An Inverse Problems Approach to Pulse Wave Analysis in the Human Brain*. 2024. arXiv: 2402.09803 [math.NA].

[24] L. Weissinger, S. Hubmer, R. Ramlau, and H. U. Voss. "An Inverse Problems Approach to Pulse Wave Analysis in the Human Brain". In: *SIAM Journal on Imaging Sciences* 18.1 (Jan. 2025), pp. 60–88. ISSN: 1936-4954. DOI: https://doi.org/10.1137/24M163921X.